

MO Conception de sites web

Département TIC



2025 – Semestre 7

Plan

- 1 Le PHP
 - PHP : prétraitement côté serveur
- 2 Formulaires : remontée d'informations
- 3 Sessions : suivi des clients
- 4 Bases de données : personnaliser le contenu
 - Mise en place d'une connexion avec MySQL
 - Se protéger des injections SQL
 - Application à la connexion d'un utilisateur
- 5 Git

Prétraitement en PHP

Le prétraitement

- Permet de réaliser des traitements “coté serveur”,
- Permet d'inclure du contenu commun à différentes pages,
- Génère du code HTML côté serveur avant l'envoi au navigateur.

Code PHP

- Dans un fichier avec l'extension **.php**,
- Ce fichier peut contenir du **HTML**,
- Le code PHP est entre les balises **<?php** et **?>**.

Exemple de code réalisé à partir du cours.

Séquence de traitements d'une ressource PHP

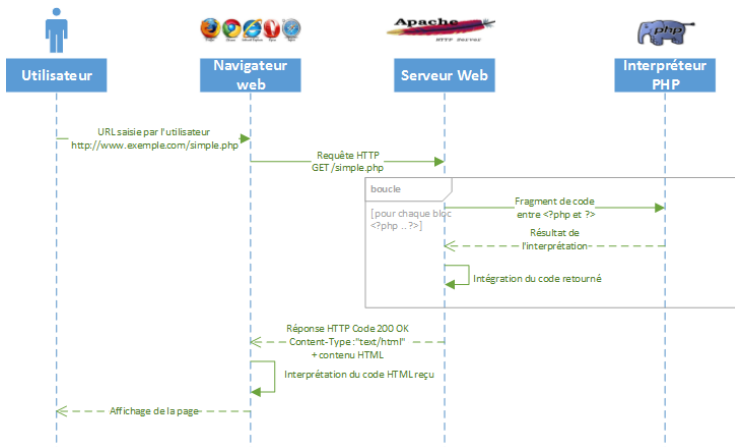


Figure – Prétraitement de la page par l'interpréteur PHP

Exemple : intégration d'éléments dynamiques

Date et heure côté serveur

```
<!-- Nous sommes en HTML -->
<p>Bonjour en HTML</p>
<?php // Début du PHP
    // echo permet "d'écrire dans le HTML"
    echo "<p>Bonjour en PHP</p>\n";
    // Déclaration d'une variable pour la date et l'heure
    // Notez le $ obligatoire au début du nom des variables :
    $maintenant = new DateTime();
    // Pour avoir l'heure en France :
    $maintenant->setTimezone(new DateTimeZone('Europe/Paris'));
    // Affichage de la date, puis de l'heure (concaténation avec le .):
    echo "<p>Nous sommes le ".$maintenant->format('d/m/Y')."</p>\n";
    echo "<p>Il est ".$maintenant->format('H:i:s')."</p>\n";
?>
```

Testez le code, et vérifiez le code source.

Composition à partir de fragments communs 1/2

Définition d'un fragment header.inc.php avec le début de page

Lors de la création d'un site, l'en-tête de toutes les pages sera identique. Il est intéressant de :

- Placer le code dans un fichier spécifique,
- Puis de l'inclure dans toutes les pages.

```
<!DOCTYPE html>
<html Lang="fr">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?php echo $titre_page; ?></title>
    <link rel="stylesheet" href="/css/bootstrap.min.css">
    <link rel="stylesheet" href="/css/font-awesome.min.css">
    <link rel="stylesheet" href="style.css">
</head>
```

Composition à partir de fragments communs 2/2

Inclusion dans la page des fragments .inc.php

```
<?php
    // Définition du titre :
    $titre_page = "Page d'accueil";
    // Inclusion de l'en-tête :
    include 'header.inc.php';
?>
<body>
    <!-- Inclusion d'un menu : : -->
    <?php include 'menu.inc.php'; ?>
    <!-- Reste de la page, par exemple : -->
```



Paramétrisation des fragments

La variable \$titre_page est

- *définie* dans la page
- *utilisée* dans le fragment

Plan

- 1 Le PHP
 - PHP : prétraitement côté serveur
- 2 Formulaires : remontée d'informations
- 3 Sessions : suivi des clients
- 4 Bases de données : personnaliser le contenu
 - Mise en place d'une connexion avec MySQL
 - Se protéger des injections SQL
 - Application à la connexion d'un utilisateur
- 5 Git

Exemple : formulaire de contact

Formulaire de contact

Votre formulaire contient des erreurs

Votre nom

Nom et prénom

Cice Kipran

Comment vous répondre

Vos coordonnées

Numéro de téléphone

+33 zero six vingt et un

Le numéro de téléphone ne doit contenir que des chiffres

Adresse de courriel

xoo@yyy.zz

Votre email ne sera pas vendu !

Votre message

Que se passe-t-il s'il n'y a pas de script au bout ?

Veuillez saisir au moins 20 caractères

Envoyer

Un formulaire

- Permet de *remonter* de l'information,
- Les données doivent être validées *côté serveur*,
- L'interface doit prévoir un retour d'information.

Les formulaires avec Bootstrap.

Demande ou envoi ?



A screenshot of the Google search homepage. The Google logo is centered at the top. Below it is a search bar containing the text "esigelec". To the right of the search bar are a close button (X) and a microphone icon. Below the search bar are two buttons: "Recherche Google" and "J'ai de la chance".

Inscription

Nom

Prénom

Email

Mot de passe

Choix de la méthode HTTP

Méthode GET

- *Demande* de documents
 - Recherche, filtrage, ...
- Passe par l'URL de la page,
- Ne doit pas modifier le contenu de la base de données.

Méthode POST

- *Envoi* de données à traiter :
 - Inscription, connexion, participation, ...
- Encodage des données dans le corps de la requête HTTP
- Peut modifier la base de données

Exemple : transmission par la méthode GET

  Recherche GoogleJ'ai de la chance

x	Headers	Preview	Response	Initiator	Timing	Cookies
▼ General						
Request URL: <code>https://www.google.fr/search?q=esigelec</code>						
Request Method: GET						
Status Code:  200						
Remote Address: 142.250.75.227:443						

Exemple : transmission par la méthode POST

Connexion réussie

Accueil

Contenu

Petite démo de ce que vous allez voir dans ce cours. Une partie du code vous est donnée sur ENT.

Elements Console Sources Network Performance Memory Application Security Lighthouse			
☒ ☐ ☐ ☐ ☐ Preserve log ☐ Disable cache No throttling ☐ ☐ ☐			
50 ms 100 ms 150 ms 200 ms 250 ms 300 ms 350 ms 400 ms 450 ms 500			
Name	Status	Type	Initiator
tt_connexion.php	302	document / Redirect	Other
index.php	200	document	tt_connexion.php
bootstrap.min.css	200	stylesheet	index.php:7
style.css	200	stylesheet	index.php:8
running_2.jpeg	200	jpeg	index.php:58
bootstrap.bundle.min.js	200	script	index.php:64
X data:image/svg+xml,...	200	svg+xml	bootstrap.min.css

Figure – Transmission dans le corps de la requête

Attributs importants d'un formulaire

Attributs de la balise <form>

`method=["get"|"post"]` méthode de requête HTTP

`action=[path|uri]` adresse du script de traitement du formulaire

Attributs de la balise <input>

`type=[text|password|checkbox|radio|submit|reset|file|hidden|image|button|...]` choix du type de contrôle

`name=[chaine]` nom de la variable à traiter par le script

`value=[chaine]` valeur de la variable envoyée au script

Contrôles sémantiques HTML 5

- HTML 5 introduit de nouveaux contrôles
- Valeurs possibles de l'attribut `type` de l'élément `input` :
 - `email` le champ requiert un contenu au format d'adresse électronique
 - `url` le champ accueille des URL valides
 - `tel` le champ est destiné aux numéros de téléphone
 - `number` le champ accepte uniquement les caractères numériques
 - `color` le champ est prévu pour les chaînes représentant une valeur de couleur
- Le navigateur peut afficher des interfaces spécifiques et des validations préventives
- Si un contrôle n'est pas reconnu, il est considéré comme `text`

Exemple : formulaire POST

Exemple de formulaire n'utilisant pas les classes *Bootstrap* :

```
<form action="traitement.php" method="post">
  <!-- Champ pour la saisie du nom : -->
  <label for="nom">Nom :</label><br>
  <input type="text" id="nom" name="le_nom" placeholder="Entrez votre nom"><br>
  <!-- Champ pour la saisie de l'email : -->
  <label for="mail">Email :</label><br>
  <input type="email" id="mail" name="l_email" required><br>
  <!-- Champ pour la saisie du mot de passe : -->
  <label for="pass">Mot de passe :</label><br>
  <input type="password" id="pass" name="le_mot_de_passe" required><br>
  <!-- Bouton pour envoyer le contenu à traitement.php : -->
  <input type="submit" value="Envoyer">
</form>
```

Testez le code.

Exemple : le fichier traitement.php

Les valeurs saisies dans le formulaire précédent sont accessibles :

```
<?php
// Le formulaire a été envoyé avec la méthode POST
// On récupère le contenu du formulaire dans $_POST
    $nom = $_POST['le_nom'];
    $email = $_POST['l_email'];
    $password = $_POST['le_mot_de_passe'];
// Ici affichage des 3 valeurs :
    echo "<p>Le nom : ".$nom."</p>";
    echo "<p>L'email : ".$email."</p>";
    echo "<p>Le mot de passe : ".$password."</p>";
?>
```

Bilan : Formulaires

- Les formulaires permettent de *remonter* de l'information,
- La *méthode* définit le type de transmission :
 - Via l'URL pour une méthode GET,
 - Dans le corps de la requête pour la méthode POST,
- Le traitement se fait en PHP par les variables `$_GET` ou `$_POST` en fonction de la méthode,
- Après le traitement on peut afficher la page contenant le script (code 200) ou rediriger vers une autre page (code 302).

Séparation

Essayez de toujours séparer le code du formulaire et le code du traitement !

Plan

- 1 Le PHP
 - PHP : prétraitement côté serveur
- 2 Formulaires : remontée d'informations
- 3 Sessions : suivi des clients
- 4 Bases de données : personnaliser le contenu
 - Mise en place d'une connexion avec MySQL
 - Se protéger des injections SQL
 - Application à la connexion d'un utilisateur
- 5 Git

Exemple : sessions PHP

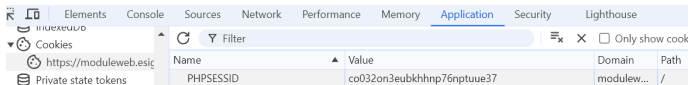
Une session

- Compense l'absence d'état dans HTTP,
- Permet de partager des données entre différentes pages, comme par exemple des informations de connexion,
- Les données sont stockées dans un tableau associatif :
\$_SESSION,
- Permet de suivre un client pendant sa navigation,
- Utilise le mécanisme de cookies.

Voir les sessions sur le site de PHP.

Cookies de sessions PHP

- Cookie : fichier texte stocké par le navigateur
 - Valeurs transmises dans la requête HTTP avec l'en-tête *Cookie*.



- Utilisation transparente en PHP :
 - `<?php session_start()?>` au début de chaque page,
 - Active le tableau associatif `$_SESSION`,
 - `$_SESSION` conserve son contenu de page en page,
- Voir le principe des sessions sur le site de PHP.

Cas d'utilisation des sessions PHP

- Authentification :
 - Un formulaire d'authentification est proposé,
 - Après vérification des données de l'utilisateur, les éléments intéressants sont conservés dans le tableau de session (email, pseudo, ...),
 - On sait ainsi sur chaque page si un utilisateur est authentifié.
- Protection des pages :
 - Si la session permet de savoir qui est authentifié, il est possible d'utiliser ces informations pour autoriser ou interdire l'accès de certaines pages,
 - Par exemple une page d'administrateur.
- Affichage pour l'utilisateur
 - Les pages *PHP* effectuant des traitements ne peuvent rien afficher,
 - On peut alors mettre dans la session un message qui sera affiché après redirection (`$_SESSION['message']`).

Plan

- 1 Le PHP
 - PHP : prétraitement côté serveur
- 2 Formulaires : remontée d'informations
- 3 Sessions : suivi des clients
- 4 Bases de données : personnaliser le contenu
 - Mise en place d'une connexion avec MySQL
 - Se protéger des injections SQL
 - Application à la connexion d'un utilisateur
- 5 Git

Bases de données

 Présentation Enseignements L'école Contact				
Quand	Qui	Téléphone	Courriel	Message
11/09/2017 05:09:16	Jean PeuxPlus	(+33) 123456789	jpp@ninetythree.com	Comment bootstrapper mon pied ?
13/09/2017 05:09:29	Gui Root	(+33) 320032003		Peut-on sécuriser un site PHP ?
13/09/2017 07:09:05	Troy Zieme	(+33)	troy@zieme.name	L'accessibilité, c'est ARIA ?
13/09/2017 12:09:19	Katre Car	(+33) 123456789		mais pourquoi donc autant de caractères ?
13/09/2017 13:09:27	Sans retour	(+33)		Je ne fournis aucun contact.

ESIGELEC : Technopôle du Madrillet
Avenue Galilée - BP 10024
76801 Saint-Etienne du Rouvray Cedex

☎ (+33) 02 32 91 58 58

✉ tenier@esigelec.fr

La base de données

- Stocke les données dynamiques du site,
- Permet de personnaliser l'expérience client, en association avec la session,
- Doit être sécurisée !

<https://php.net/manual/fr/security.database.php>

Choix d'une extension PHP/MySQL

Extension mysql

- Obsolète (depuis PHP 5.5)
- Interface procédurale
- *Non disponible* sur le serveur de production

Extension mysqli

- Évolution de l'extension mysql
- Interface procédurale ET objet
- API spécifique à MySQL

Extension PDO

- Interface objet uniquement
- Reprend le principe de JDBC pour JAVA
- API commune à tous les SGBD

<http://php.net/manual/fr/mysqliinfo.api.choosing.php>

Comparaison mysqli / PDO

Exemple de connexion et de requête simple :

mysqli

```
1 $mysqli = new mysqli("example.com", "user", "password", "madb");
2 $res = $mysqli->query("SELECT attribut FROM matable");
3 $row = $res->fetch_assoc(); //première ligne du résultat
4 echo htmlentities($row['attribut']); //affichage d'un attribut
```

PDO

```
1 $pdo= new PDO("mysql:host=example.com;dbname=madb","user","password");
2 $res= $pdo->query("SELECT attribut FROM matable");
3 $row = $res->fetch(PDO::FETCH_ASSOC); //première ligne du résultat
4 echo htmlentities($row['attribut']); //affichage d'un attribut
```

Exemple de configuration

Configuration du serveur de développement

- Création de 4 variables PHP dans un fichier d'inclusion,
- Récupération des variables dans chaque script utilisant la BDD :
`<?php require_once("param.inc.php"); ?>`
- Le déploiement sur le serveur de production sera facilité.

param.inc.php

```
1 <?php
2     $host='localhost';
3     $login='root';
4     $password='root';
5     $dbname='madb';
6 ?>
```

Sur le serveur

- Attention, la valeur des variables devra être modifiée sur le serveur de production,
- Vos paramètres sur le serveur vous seront donnés la prochaine fois.

Exemple de page générée depuis une base de données

Affichage généré depuis la table *clients* d'une base *commerciale*

```
<body>
  <h1>Noms des clients :</h1>
  <?php
    require_once('param.inc.php');
    $mysqli = new mysqli($host, $login, $password, $dbname);
    if($mysqli->connect_errno) {
      echo "Echec : ".$mysqli->connect_error." (". $mysqli->connect_errno.")";
    } else {
      $resultat=$mysqli->query("SELECT * FROM clients");
      if(!$resultat) {
        die("Echec de la requête : ".$mysqli->error);
      } elseif($resultat->num_rows == 0) {
        echo "<p>Aucun résultat</p>";
      } else {
        while($tuple = $resultat->fetch_assoc()) {
          echo "<p>".$tuple['nom']."</p>";
        }
      }
    }
  <?>
</body>
```

Points clés de la récupération des données

❶ Connexion à la base de données

- `$mysqli = new mysqli($host,$login,$password,$dbname);`
- Peut échouer : toujours tester si `$mysqli` vaut **FALSE**

❷ Envoi de la requête

- `$res=$mysqli->query("SELECT * FROM clients");`
- `$res` contient le résultat de la requête ou **FALSE**
 - En cas d'échec, afficher `$mysqli->error` pour identifier le problème

❸ Récupération du résultat de la requête dans un tableau

- Il faut appeler `$res->fetch_assoc()` autant de fois qu'il y a de lignes retournées par la requête SQL
- `fetch_assoc()` retourne un tableau associatif dont les clés sont chaque attribut demandé dans le **SELECT**

Règles de sécurité

- ❶ Toujours valider tous les éléments saisis par l'utilisateur
- ❷ Échapper les caractères avec :
 - mysqli : `real_escape_string`
 - PDO : `quote`
- ❸ Privilégier les requêtes préparées
- ❹ Réduire les privilèges de la connexion SQL
 - Exemple : Le serveur de production ne vous donne pas accès aux commandes de création/suppression de bases

Exemple d'attaque

Attaque par injection SQL

- Insertion par l'utilisateur de code SQL dans un champ de formulaire
- Objectifs
 - 1 Accéder à des données
 - 2 Modifier des données
 - 3 Se connecter sans mot de passe
- Exemple de code vulnérable

```
1 $input=$_POST['nom'];//aucun filtrage ni validation !!
2 $password=$_POST['password'];
3 $res=mysqli->query("SELECT userid FROM users WHERE username='
    $input' AND password='$password'");
```

- L'injection est réalisée en saisissant 'OR 1=1 --' dans le champ de formulaire ayant l'attribut `name="nom"`

Protection par utilisation des requêtes préparées

2 étapes : préparation et exécution

Préparation d'une requête

```
1 if (!($stmt = $mysqli->prepare("SELECT userid FROM users WHERE username  
    =? AND password=?"))) {  
2     echo "Echec de la préparation : (" . $mysqli->errno . ") " . $mysqli->  
        error;  
3 }  
4 $stmt->bind_param('ss', $input, $password);
```

Exécution de la requête

```
1 //affectation  
2 $input=$_POST['nom'];  
3 $password=$_POST['password'];  
4 //execution  
5 if (!$stmt->execute()) {  
6     echo "Echec lors de l'exécution de la requête : (" . $stmt->  
        errno . ") " . $stmt->error;  
7 }  
8
```


Cryptage du mot de passe

❶ Lors de la création du compte :

- Le mot de passe doit être crypté avant d'être enregistré dans la base de données (password_hash),
- Il est préférable que l'algorithme utilisé ne permette pas de le décrypter,
- N'utilisez pas de 'hachage' direct, utilisez par exemple l'algorithme *Blowfish*.

❷ Lors de la connexion :

- Récupérez le mot de passe crypté de la base de données,
- Utilisez une fonction qui va comparer la saisie utilisateur avec ce mot de passe crypté (password_verify),
- Si les identifiants ne sont pas corrects, renvoyez l'utilisateur au formulaire de connexion en affichant un message d'erreur.

Cryptage du mot de passe

① Le 'hachage' :

- Les algorithmes de hachage ne sont pas réversibles : il n'est donc pas possible de décrypter un mot de passe crypté,
- **Problème** : On obtient toujours le même résultat en faisant le hachage d'un même mot de passe,
- Il existe des dictionnaires comportant de nombreuses entrées {mot de passe : mot de passe haché}. Un attaquant qui récupère des mots de passe hachés peut utiliser un dictionnaire pour retrouver les mots de passe non hachés.

② Avec '*Blowfish*' :

- Dans *Blowfish* on ajoute une valeur aléatoire (le '*salt*') à chaque fois que l'on crypte,
- Pour un même mot de passe, on obtient donc une valeur différente à chaque fois,
- Il n'est pas possible de faire une attaque par dictionnaire.

Exemple avec *Blowfish*

Documentation

- ❶ Déterminer si une variable est définie :
 - Fonction `isset`
 - Exemple : `if(isset($_SESSION['nom']))...`
- ❷ Supprimer une variable :
 - Fonction `unset`
 - Exemple : `unset($_SESSION['nom']);`
- ❸ Stocker et vérifier le hash d'un mot de passe dans la BDD :
 - `password_hash` (utilisez de préférence `bcrypt`),
 - `password_verify`
- ❹ Écrire l'en-tête HTTP pour rediriger l'utilisateur :
 - Fonction `header`
 - Exemple : `header('Location: index.php');`

Objectif pour aujourd'hui : connexion et déconnexion

- ❶ Mettre en place la table nécessaire dans la BDD
 - Pensez à remplir la table avec un ou deux utilisateurs. Utilisez la fonction `password_hash` pour sécuriser le mot de passe.
- ❷ Écrire le script de connexion en suivant l'algorithme proposé
 - Ce script effectue un *traitement* et *redirige vers une autre page* (code 302)
 - Il ne contient donc pas de HTML
- ❸ Implémenter la déconnexion

Plan

- 1 Le PHP
 - PHP : prétraitement côté serveur
- 2 Formulaires : remontée d'informations
- 3 Sessions : suivi des clients
- 4 Bases de données : personnaliser le contenu
 - Mise en place d'une connexion avec MySQL
 - Se protéger des injections SQL
 - Application à la connexion d'un utilisateur
- 5 Git

Principe de Git

- Git est un logiciel de gestion de version,
- Il permet de travailler facilement sur un même projet à plusieurs,
- Les personnes enregistrées sur le projet peuvent participer à distance,
- Lors de la réalisation, il est possible de placer des jalons dans le projet (commits),
- Il est possible à tout moment de revenir à un jalon.

Utilisation de Git

- Pour utiliser Git il faut d'abord l'installer sur votre ordinateur,
- L'adresse est <https://git-scm.com/>
- Il faut également avoir un compte sur un serveur,
- Pour le projet, nous utiliserons GitLab,
- <https://about.gitlab.com/>,
- Chacun d'entre vous devra y créer un compte, si possible avec l'adresse de l'école.

Utilisation de Git

Nous n'utiliserons que quelques commandes de Git.

- **clone** : Permet de faire une copie du projet sur votre ordinateur,
- **commit** : Permet de mettre un jalon dans le projet,
- **push** : Permet de mettre à jour le projet sur le serveur,
- **pull** : Permet de mettre à jour le projet sur votre ordinateur (après un push de votre binôme).

Principe

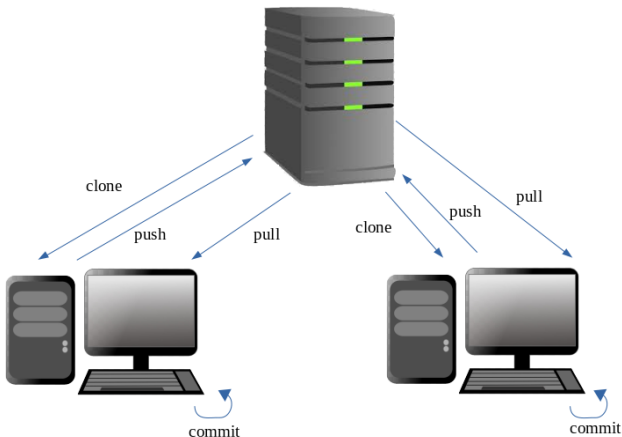


Figure – Principe de Git